Vision-Language Fashion Retrieval Using BLIP and CLIP

Blake Denson · CIS 530 Term Project · Computer Vision / Perception Track

Abstract

This report presents a prototype multimodal fashion retrieval system that uses pretrained vision-language foundation models to retrieve visually and semantically similar clothing items from a real-world fashion image dataset. The system combines BLIP for image-to-text caption generation and CLIP for embedding-based similarity retrieval. The project began as a conventional CNN image-classification idea, but the design shifted toward semantic retrieval after evaluating the actual application need: users often have a clothing image rather than a class label or exact product name. The implemented prototype uses a 1,000-image subset of the Fashion Product Images Small dataset, precomputes CLIP embeddings offline, and performs online retrieval with cosine similarity. Preliminary results show that the system generates meaningful clothing descriptions and retrieves semantically related fashion items, while precomputed embeddings reduce startup time from approximately 1–3 minutes to 5–10 seconds.

1. Introduction

The target application is visual fashion search: a user uploads an image of a clothing item and receives similar items from a dataset. This task differs fundamentally from standard image classification. A classifier predicts one fixed label—shirt, shoe, or jacket—whereas a retrieval system must rank many candidate items by visual and semantic similarity. This distinction matters because fashion recommendation is rarely about assigning a single category; it is about finding items that match shape, garment type, style, color, and user intent.

The project is aligned with the Perception and Agents category track and the Computer Vision methodological pillar. The design process evaluated several methodological focuses:

- Feature selection: choosing which metadata fields and embedding layers to use
- Feature construction: captioning via BLIP and embedding via CLIP vision encoder
- Hyperparameter optimization: top-k threshold, caption length, similarity cutoff
- Scalable vector search: brute-force cosine vs. FAISS-indexed retrieval

The final scope focuses on feature construction and semantic retrieval because those components most directly support the use case. The formal objective is to map an uploaded query image q into an embedding vector z_q and retrieve the top-k dataset embeddings z_i that maximize cosine similarity: s(q,i) = cos(z_q, z_i), returning arg top-k_i s(q,i). This objective is not optimized by end-to-end training in the current prototype; instead, transfer learning from pretrained CLIP and BLIP models provides the image and language representations, and the project contribution is the integrated retrieval pipeline, preprocessing decisions, and scalable architecture.

2. Background and Related Work

Traditional convolutional neural networks (CNNs) are strong baselines for computer vision tasks. CNNs have historically performed well on supervised image classification benchmarks such as Fashion-MNIST—a dataset of 28×28 grayscale images across 10 standardized clothing categories useful for evaluating introductory classification architectures (Xiao et al., arXiv, 2017[1]). However, Fashion-MNIST is less suitable for semantic fashion retrieval due to its simplified image structure, restricted category space, and lack of realistic product imagery or fine-grained metadata. Larger datasets such as DeepFashion and DeepFashion2 better support realistic retrieval-oriented tasks—clothing recognition, landmark detection, parsing, and semantic retrieval—but introduce significantly larger preprocessing and computational requirements (Liu et al., CVPR, 2016[2]).

CLIP is central to the implemented retrieval system. Radford et al. (ICML, 2021[3]) introduced a contrastive image-text pretraining framework in which matching image-caption pairs are pulled closer together in embedding space while mismatched pairs are pushed apart. This enables zero-shot transfer and semantic retrieval because visually or conceptually related items occupy nearby regions in embedding space. BLIP is used for the captioning stage. Li et al. (ICML, 2022[4]) proposed a bootstrapped vision-language pretraining framework capable of both image understanding and language generation, including captioning and retrieval. In this project, BLIP generates semantic clothing descriptions while CLIP produces embedding vectors used for cosine similarity retrieval.

The implementation uses HuggingFace Transformers for pretrained model loading and PyTorch for inference execution. Streamlit provides a lightweight interactive interface for image upload and retrieval visualization. Alternative implementations considered included CNN-

only retrieval systems, Vision Transformers (ViT), DeepFashion retrieval pipelines, and FAISS-based vector indexing systems available through HuggingFace Model Hub, GitHub research repositories, and Google Colab notebooks. Future deployments could expose the retrieval pipeline through REST APIs or self-hosted inference endpoints for scalable production integration. FAISS (Johnson et al., IEEE BigData, 2017[5]), a similarity-search library developed by Meta AI, would replace brute-force cosine comparisons with indexed nearest-neighbor retrieval optimized for large embedding collections.

3. Data and Descriptive Statistics

The selected dataset is the Fashion Product Images Small dataset from Kaggle (Aggarwal, Kaggle[6]). This dataset was chosen because it provides real product images and metadata including article type, color, product display name, and usage. It is more realistic than Fashion-MNIST but more manageable than DeepFashion for rapid prototyping. The prototype uses a 1,000-image subset to keep inference and preprocessing feasible on a local machine.

Field	Prototype Value
Dataset	Fashion Product Images Small (Kaggle)
Subset size	1,000 images
Input modality	RGB fashion product images
Metadata used	id, articleType, baseColour, usage, productDisplayName
Model inputs	Uploaded query image; dataset images for retrieval
Ground truth (prototype)	Qualitative relevance via semantic similarity inspection of retrieved items; formal Recall@K planned
Other datasets considered	Fashion-MNIST (Xiao et al.), DeepFashion / DeepFashion2 (Liu et al.)

Table 1. Dataset and prototype descriptive statistics.

Ground truth in this prototype was defined qualitatively through semantic similarity inspection of retrieved clothing items rather than strict category-label matching. Product retrieval datasets often require formal relevance judgments: which items count as correct matches for a given query? In a future experimental design, relevance would be defined through metadata constraints—same articleType and baseColour—enabling computation of Recall@K and Precision@K. This gap between qualitative inspection and formal metric computation is a known limitation of the current prototype and is addressed in the evaluation plan below.

The selected 1,000-image subset was sufficient for demonstrating proof-of-concept retrieval behavior and validating the multimodal pipeline architecture. However, larger datasets such as DeepFashion would likely improve semantic diversity, retrieval robustness, and generalization performance. Future work could incorporate active learning approaches to prioritize difficult or ambiguous clothing examples during dataset expansion, directly addressing the question of whether the current subset is sufficient for production-scale deployment.

4. Methodology

The baseline system is CLIP-only image retrieval. Each dataset image is encoded with CLIP's vision encoder, the uploaded query image is encoded with the same encoder, and cosine similarity ranks the dataset candidates. This baseline is simple, fast, and appropriate because it directly measures whether CLIP's image embedding space alone is sufficient for clothing similarity search.

The alternative system adds BLIP caption generation. BLIP generates a natural-language description of the uploaded clothing item—for example, 'a black polo shirt with a red and white stripe on the collar.' This caption is displayed to the user and supplies interpretability for the retrieval process. CLIP remains responsible for visual embedding and similarity search. The architecture therefore combines vision-to-text understanding with embedding-based nearest-neighbor retrieval.

Offline: precompute dataset embeddings
Online: upload image → caption → embedding → similarity search → recommendations

Figure 1. BLIP + CLIP retrieval pipeline with offline precomputation and online query stages.

The most important engineering decision was offline embedding precomputation. Initially, the app attempted to compute embeddings for all dataset images at startup, which became slow as the dataset grew. The final architecture moves dataset encoding into a dedicated preprocessing script (precompute_embeddings.py), which writes normalized image embeddings, image paths, and metadata to embeddings.pkl. The Streamlit app then loads the saved vectors and computes only the query embedding

at runtime. Normalization is critical: cosine similarity compares vector direction, so raw magnitude must be removed before comparison.

Images were resized and normalized prior to embedding generation to ensure consistent tensor dimensions and stable inference behavior across the retrieval pipeline. Hyperparameter choices were intentionally conservative: a 1,000-image subset to keep preprocessing feasible on a laptop, top-k = 5 for visual inspection, and short BLIP caption length for compact clothing descriptions.

5. Experiment Design and Results

5.1 Statistical Hypotheses

To evaluate whether the multimodal pipeline provides meaningful improvement over the baseline, the following hypotheses are stated:

H_0: The multimodal BLIP+CLIP retrieval pipeline does not significantly improve retrieval quality or runtime performance compared to the CLIP-only baseline.

H_1: The multimodal BLIP+CLIP retrieval pipeline improves semantic retrieval quality and retrieval efficiency compared to the CLIP-only baseline.

$\alpha = 0.05$ (significance level). A practical evaluation would test a fixed set of query images with both systems and compare per-query Recall@K. Because the same queries are evaluated by both systems, a paired statistical test or bootstrap confidence interval over queries is more appropriate than treating samples as independent.

5.2 Validation Strategy

Because the project focused primarily on retrieval quality rather than supervised classification accuracy, bootstrap sampling was selected over k-fold cross-validation. Bootstrap resampling allows repeated evaluation of retrieval behavior under small-dataset conditions while preserving flexibility in the embedding retrieval pipeline. In a future formal evaluation, 30 bootstrap samples would be drawn from the available query set to estimate confidence intervals around Recall@K and Precision@K, with each sample testing the full top-k ranking on a held-out query image removed from the retrieval index.

5.3 Evaluation Metrics

Traditional classification metrics such as AUROC and F1-score are less directly applicable because the project focuses on semantic retrieval rather than discrete label prediction. The appropriate formal metrics for a retrieval system are:

- Recall@K: whether at least one relevant item appears within the top-K retrieved results
- Precision@K: fraction of top-K results that are relevant
- Mean Reciprocal Rank (MRR): average of the reciprocal of the rank of the first relevant item
- Retrieval latency (wall-clock time): measured at startup and per-query stages

In the current prototype, cosine similarity scores serve as the proxy metric pending formal relevance labeling. Future evaluation would define relevance through metadata constraints (matching articleType and baseColour) or human judgment to compute the above metrics over a held-out query set.

5.4 Runtime Results

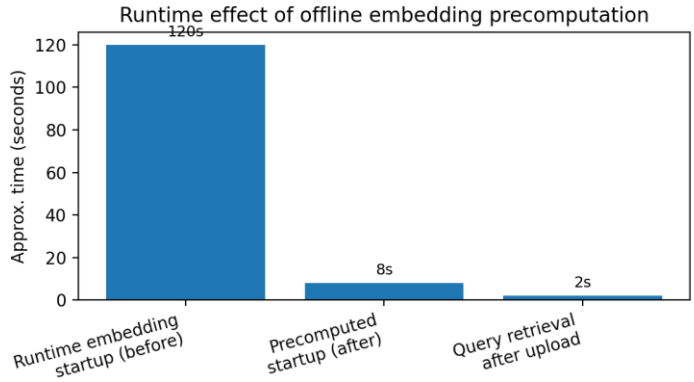

Figure 2. Wall-clock runtime comparison: live embedding generation vs. precomputed embedding loading vs. per-query retrieval latency.

The runtime comparison demonstrates the practical benefit of offline precomputation. Live embedding generation at startup required approximately 60–180 seconds for a 1,000-image dataset on a local laptop (measured as wall-clock time). Loading precomputed embeddings from embeddings.pkl required approximately 5–10 seconds. Per-query retrieval after image upload required approximately 1–3 seconds. These measurements show that offline precomputation reduces startup latency by approximately one order of magnitude, directly improving interactive usability without requiring a specialized vector database.

5.5 Retrieval Quality Results

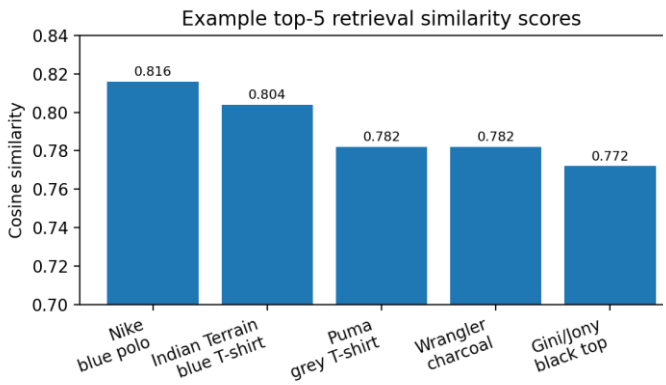

Figure 3. Top-5 cosine similarity scores from a representative black polo shirt query.

A representative query used a black polo shirt image. BLIP generated the caption 'a black polo shirt with a red and white stripe on the collar.' The top retrieved results were primarily polos, T-shirts, and tops with similar garment structure. Similarity scores ranged from approximately 0.772 to 0.816 across the top-5 results. Items were semantically related clothing items rather than exact product matches, which is the intended behavior for a retrieval prototype operating without labeled relevance judgments.

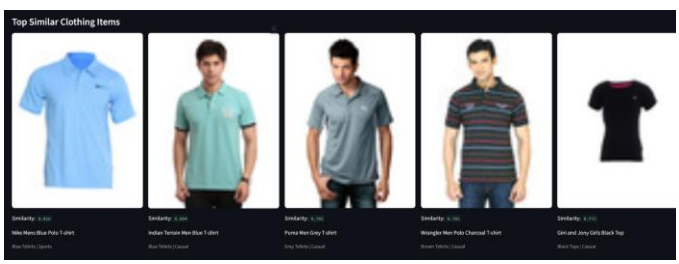

Figure 4. Streamlit retrieval interface showing top-5 similar clothing items and cosine similarity scores for a sample query.

System	Captioning	Retrieval Method	Strength
CLIP-only baseline	No	Cosine similarity on CLIP image embeddings	Simple, fast semantic image retrieval
BLIP + CLIP (proposed)	Yes (BLIP)	BLIP caption + CLIP image embedding + cosine similarity	Interpretable multimodal pipeline with semantic descriptions
Future FAISS version	Optional	Indexed nearest-neighbor vector search	Scales to large datasets; sub-linear retrieval time

Table 2. Baseline, proposed system, and scalable future architecture comparison.

6. Error Analysis and Statistical Evaluation Plan

The current system was not trained end-to-end, so traditional training loss curves are not applicable. The relevant empirical questions concern retrieval quality, runtime, and robustness. A rigorous evaluation would sample query images from the dataset, remove each query from the retrieval index, and test whether the system retrieves items with matching articleType and baseColour within the top K. This produces Recall@1, Recall@5, Recall@10, and Precision@K scores over the query set.

For statistical testing, a paired comparison would evaluate CLIP-only retrieval against BLIP+CLIP retrieval on the same fixed query set. Under H_0, the two systems produce equivalent Recall@K. Bootstrap confidence intervals over query samples or a paired Wilcoxon signed-rank test over per-query outcomes would support a more rigorous claim at $\alpha = 0.05$. An ablation study would remove BLIP captioning, remove metadata display, or adjust similarity thresholds to isolate each component's contribution.

The strongest observed limitation is that high cosine similarity does not guarantee user satisfaction. A retrieved item can be semantically close in embedding space but still differ in color, brand, or style preference. This metric-divergence problem—where the proxy metric measures embedding closeness while the practical objective is useful recommendation—motivates future evaluation with user-centered relevance rubrics, category-specific filtering, and ablation studies comparing CLIP-only against BLIP+CLIP retrieval on formal Recall@K.

7. Summary and Future Work

This project produced a working multimodal retrieval prototype combining BLIP caption generation, CLIP semantic image embeddings, cosine similarity search, and offline embedding precomputation. The project evolved from CNN classification into a more appropriate retrieval architecture after clarifying the actual application need. The final prototype demonstrates that pretrained vision-language models can support semantic fashion search without training a new model from scratch.

Future work should focus on four directions. First, the dataset should be expanded beyond the 1,000-image subset, potentially using DeepFashion or DeepFashion2, with active learning approaches to prioritize difficult or ambiguous examples during expansion. Second, brute-

force cosine similarity should be replaced with FAISS or another vector database for scalable retrieval. Third, evaluation should use formal Recall@K and Precision@K metrics with statistically valid bootstrap confidence intervals over a fixed query set. Fourth, the system could be extended into trend analytics: a captioner can process longitudinal product or point-of-sale imagery over time to track changes in colors, garment types, and styles across daily or weekly windows—for example, detecting seasonal shifts in dominant hues or garment categories at the product level.

The long-term goal is a fashion discovery platform that accepts images, generates semantic descriptions, retrieves visually related items, and links users to similar products through web APIs or scraping pipelines. Pre-trained or fine-tuned CV models available on HuggingFace could further enhance retrieval quality through domain-specific fine-tuning on DeepFashion annotations, or through inference-only access on free commercial platforms.

8. Implementation Details and Design Alternatives

The implemented codebase contains two main scripts. The Streamlit application (app.py) loads BLIP, CLIP, the precomputed embedding file, and product metadata for display. The preprocessing script (precompute_embeddings.py) loads dataset images, encodes them through the CLIP vision encoder, normalizes the resulting vectors, and stores them with image paths and metadata in embeddings.pkl. Normalization ensures that cosine similarity reflects directional agreement rather than raw vector magnitude.

Several alternative design paths were considered. A CNN classifier would be easier to evaluate with accuracy metrics but would not solve the core product-search use case because it predicts a closed class label rather than returning visually similar items. Fashion-MNIST was considered as an early benchmark but is too small, grayscale, and simplified for product retrieval. DeepFashion was considered for its richer annotations but its dataset scale and annotation complexity made it less appropriate for a first working prototype. Alternative implementations surveyed included Vision Transformer (ViT) retrieval systems, DeepFashion retrieval pipelines, and FAISS-based systems available through HuggingFace, GitHub research repositories, and Google Colab notebooks.

Hyperparameter choices were intentionally conservative: a 1,000-image subset to keep preprocessing manageable on a laptop; top-k = 5 for visual inspection; short maximum BLIP output length for compact clothing descriptions; cosine similarity because CLIP embeddings are designed for normalized vector comparison.

9. Performance Measures and Objective Function

The optimization objective of the retrieval system differs from the performance measures shown to the user. The system is not trained during the project, so it does not minimize cross-entropy loss or mean squared error. The operational objective is ranking: for a query image, assign higher similarity scores to relevant dataset images than to irrelevant ones. Cosine similarity is the scoring function used at inference time—a proxy for semantic relevance, not a perfect measure of user satisfaction.

The final evaluation separates loss, score, and real-world utility. Cosine similarity is the retrieval score. Recall@K and Precision@K summarize whether relevant items appear near the top of the ranked list. User satisfaction, affordability, availability, brand preference, and style preference are practical objectives not captured by cosine similarity alone. This gap motivates future work on metadata filters, user-preference modeling, and purchase-link integration. The null hypothesis H_0 posits no improvement from the BLIP+CLIP pipeline over the CLIP-only baseline; a practical evaluation with paired bootstrap testing at $\alpha = 0.05$ over a fixed query set would resolve this.

10. Reproducibility and Generative AI Use

The GitHub repository is the single source of truth for the project code. It contains the Streamlit application, the preprocessing script, requirements.txt, and documentation. The large dataset and serialized embedding file are intentionally excluded from Git because they are too large and can be regenerated using the preprocessing script. This keeps the repository lightweight while preserving reproducibility.

Generative AI was used as an exoskeleton for brainstorming, debugging, and explaining technical concepts. Key AI-assisted interactions included understanding embeddings, cosine similarity, BLIP, CLIP, transfer learning, FAISS, and the difference between classification and retrieval. AI was also used to debug model-output issues. The final design choices, dataset selection, and interpretation were guided by project requirements and instructor feedback rather than delegated wholesale to an AI system.

The main threats to reproducibility are local hardware differences, model download availability, and dataset path differences. Runtime numbers are reported as approximate laptop observations rather than controlled benchmark measurements. A rigorous reproduction package would include a fixed query set, a deterministic subset of dataset IDs, a saved evaluation script, and a formal metric report.

References

[1] Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. arXiv.

[2] Liu, Z., Luo, P., Qiu, S., Wang, X., & Tang, X. (2016). DeepFashion: Powering robust clothes recognition and retrieval with rich annotations. CVPR.

[3] Radford, A., Kim, J. W., Hallacy, C., et al. (2021). Learning transferable visual models from natural language supervision. ICML.

[4] Li, J., Li, D., Xiong, C., & Hoi, S. (2022). BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation. ICML.

[5] Johnson, J., Douze, M., & Jégou, H. (2017). Billion-scale similarity search with GPUs. IEEE BigData. (FAISS)

[6] Aggarwal, P. Fashion Product Images Small Dataset. Kaggle. https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images-small